

CSCE 636 DEEP LEARNING · MODULE 04 · LECTURE NOTES

Regularisation and Normalisation

Two different jobs that are routinely confused.

MODULE OUTCOMES

- Apply dropout and explain what it actually does.
- Explain batch normalisation and why the original explanation was wrong.
- Choose between batch, layer, and group normalisation.
- Explain augmentation as the strongest regulariser available.
- Diagnose which problem you have.

1 Two different problems

NORMALISATION MAKES IT TRAINABLE; REGULARISATION MAKES IT GENERALISE

Normalisation addresses an *optimisation* problem: activations drifting in scale as they propagate through depth, which makes the loss surface badly conditioned and the gradients unreliable (Module 02 §3). It is about whether training works at all.

Regularisation addresses a *generalisation* problem: the gap between training and test error (CSCE 633 Module 03 §1). It is about whether a model that has trained successfully is any use.

The two are routinely conflated, partly because batch normalisation happens to provide a little of both, and partly because in practice both are 'things you add to the model to make it work' and get reached for in the same mood.

Diagnose before prescribing. A large train–test gap is a regularisation problem; a model whose loss will not move at all is an optimisation problem, and the remedies do not transfer. Adding dropout to a model that cannot train makes it worse, and adding batch norm to an overfitting model mostly does nothing for the gap.

2 Dropout

```

TRAINING  zero each unit independently with probability p,
           then scale the survivors by 1/(1-p)
INFERENCE do nothing

```

WHY THE SCALING ("inverted dropout")
 without it, the expected activation at inference is $1/(1-p)$ times larger than during training, so every downstream layer sees a different input distribution than it was trained on. Scaling during TRAINING keeps the expectation matched and leaves inference with no special case at all.

WHY IT HELPS

- * no unit can rely on a specific other unit being present, so fragile co-adapted detectors cannot form
- * it approximates averaging an ensemble of 2^n weight-sharing subnetworks

WHERE IT IS USED NOW
 standard in fully connected layers and in transformers.
 LARGELY ABANDONED in CNNs -- batch norm and augmentation regularise better, and dropout interacts badly with batch norm's running statistics.

The train/inference distinction is the classic dropout bug — leaving dropout active at evaluation, or forgetting to call the framework's eval mode. **It produces a model that is quietly worse rather than visibly broken**, which is the worst kind of bug: the numbers are plausible and wrong. The batch-norm interaction is also real — dropout changes the variance of the activations between training and inference, which shifts the statistics batch norm accumulated, and the two together can underperform either alone.

3 Normalisation

THE ORIGINAL EXPLANATION WAS WRONG, AND IT STILL WORKS

The original claim was that batch normalisation reduces 'internal covariate shift' — the change in each layer's input distribution as the layers beneath it update during training. It is an intuitive story and it was accepted for several years.

It was later tested directly and did not hold. Santurkar and colleagues injected deliberate noise *after* the batch norm layer, restoring and worsening the covariate shift — **and the benefit of batch norm remained.** The proposed mechanism was not the operative one.

The current account is that it smooths the loss surface, reducing the Lipschitz constant of the loss and its gradient, which makes gradients more predictive over longer distances and permits much larger learning rates (Module 03 §1). That account has better evidence and is not the final word either.

So a technique with an incorrect justification became universal because it demonstrably worked. **That is worth noting about this field:** the empirical result led, the explanation followed, and the explanation was wrong for three years while the technique was in every network. **It is a reason to hold mechanistic stories loosely and measurements tightly** — which is Module 03 §4's position too.

Type	Normalises over	Use when
Batch norm	The batch dimension, separately per channel.	Convolutional networks with batch size 32 or more. Breaks down at small batch sizes, where the statistics are noisy.
Layer norm	The feature dimension, separately per example.	Transformers and recurrent networks. Batch-independent , so training and inference are identical and batch size does not matter — which is why it took over.
Group norm	Groups of channels, per example.	Convolutional networks that must use small batches — object detection, segmentation, 3D volumes, anything memory-bound.
Instance norm	Each channel separately, per example.	Style transfer, where removing per-image contrast and colour statistics is exactly the desired effect.
RMSNorm	Scale only — no mean subtraction.	Cheaper than layer norm and works about as well; common in large language models where the saving is material.

BATCH NORM BEHAVES DIFFERENTLY AT INFERENCE

During training it normalises using the current mini-batch's mean and variance. At inference it uses a running average of those statistics, accumulated over training.

So the function computed is genuinely different in the two modes — and forgetting to switch modes is among the most common bugs in deep learning, producing results that are wrong by a plausible-looking margin.

And if the running statistics have not converged, inference is wrong even with entirely correct code — which happens with short training runs, with an unusual data ordering, or when the momentum on the running average is badly set.

More fundamentally, it makes a training example's output depend on the *other examples in its batch*, which is a genuinely strange property for a model to have: the prediction for one input is not a function of that input alone. It is the deepest reason layer norm is preferred wherever batch size varies or batch composition is not controlled, and it causes real difficulties in distributed training (Module 08) where the batch is split across devices.

4 Augmentation

AUGMENTATION ENCODES INVARIANCES YOU ALREADY KNOW

A horizontally flipped photograph of a cat is still a photograph of a cat. Training on both the image and its flip tells the model an invariance it would otherwise have to infer from examples — and inferring it costs data.

That is precisely the mechanism of Module 01 §2 — encoded prior knowledge substituting for examples — **applied through the data rather than through the architecture.** Convolution builds translation invariance into the model; augmentation builds other invariances into the training distribution, and the two are alternatives for the same job.

And it is usually the single largest win available when data is limited, well ahead of any amount of dropout or weight decay. Modern augmentation policies — RandAugment, mixup, CutMix — are a substantial part of why vision models improved without architectural change.

But the invariance must actually be true. A vertically flipped '6' is a '9'; a mirrored chest radiograph depicts dextrocardia, which is a real and rare condition; a colour-jittered image of a banana may no longer be ripe. **A false augmentation teaches the model something wrong**, and it does so invisibly, degrading exactly the cases where the invariance fails.

- **The loss will not move from its initial value:** an optimisation problem. Check initialisation, learning rate, and normalisation (Module 03 §4), and run the overfit-one-batch test before anything else.
- **Trains well, large train–test gap:** a generalisation problem. **Augmentation first** (it is usually the largest effect), then weight decay, then dropout, **and more data if it is available** (CSCE 633 Module 03 §1).
- **Trains, and both errors are high:** underfitting. More capacity, better features, or longer training — **and remove regularisation**, which is a legitimate move that is rarely considered because adding things feels like progress.
- **Results vary wildly between runs with the same configuration:** a learning rate or initialisation problem, not a regularisation one. **Run the same config three times before concluding anything from a single run.**
- **And add one thing at a time, measuring each. Stacking four regularisers simultaneously tells you nothing about which helped** — and in this field the answer is frequently 'none of them, and the fourth one hurt'.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Srivastava et al. — Dropout (free)	The §2 method and the ensemble argument, from the original.

Resource	What it covers for this module
Santurkar, Tsipras, Ilyas & Madry — How Does Batch Normalization Help Optimization? (free)	The §3 correction. The noise-injection experiment is the part to read — it is a model of how to test a mechanistic claim.
Wu & He — Group Normalization (free)	The small-batch problem of §3, measured, with the alternative.
Cubuk et al. — RandAugment (free)	The §4 practice at its current state, with the search space reduced to two hyperparameters.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Implement inverted dropout** and verify the expected activation matches between train and inference.
- 2 **Leave dropout active at inference** and measure how much worse the model is.
- 3 Implement batch norm from scratch, including the running statistics.
- 4 **Evaluate without switching to eval mode** and report the difference.
- 5 **Train with batch size 2** and compare batch norm against group norm.
- 6 Compare batch, layer, and RMS normalisation on the same transformer block.
- 7 **Measure the effect of augmentation alone** against dropout alone against weight decay alone, on the same train–test gap.
- 8 **Apply a false augmentation** (vertical flip on digits) and measure the damage.
- 9 Stack four regularisers at once, then ablate them one at a time, and report which actually helped.
- 10 Run the same configuration three times and report the spread.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Distinguish normalisation from regularisation and say how to diagnose which you need.
- 2 Why does inverted dropout scale during training?
- 3 Give two accounts of why dropout helps.
- 4 What was batch norm's original explanation, and how was it refuted?
- 5 Compare five normalisation variants.
- 6 Give three problems caused by batch norm's batch dependence.
- 7 Why does augmentation work, and what is the mechanism it shares with architecture?

- 8 Give an example of a false augmentation.
- 9 Give five diagnoses and the response to each.